

WILP, BITS PILANI DEEP NEURAL NETWORKS (AIML ZG511)

Session 11: Seq2Seq Architectures & Attention Mechanisms

Encoder-Decoder Bottlenecks, Additive vs Multiplicative Attention, and Soft Alignment

Module 8

Session 11 Reader
Full Worked Solutions

1 Sequence-to-Sequence (Seq2Seq) & The Bottleneck Problem

Seq2Seq networks encode a whole input sequence into a single vector, causing a major memory bottleneck.

Standard sequence-to-sequence (Seq2Seq) architectures consist of an Encoder RNN and a Decoder RNN. The Encoder reads input sequence $(x_1, x_2, \dots, x_T)$ and compresses it into a single context vector $c = h_T$. The Decoder generates target sequence $(y_1, y_2, \dots, y_{T'})$ from c .

Figure 1: The Seq2Seq Encoder-Decoder bottleneck problem. All input context is squeezed into a single fixed vector c .

The Interpreter's Limit

Imagine listening to a 20-minute speech in French, taking no notes, and then translating the whole speech into English strictly from memory. Long sentences get forgotten!

2 The Attention Mechanism: Soft Alignment

Attention lets the decoder look back at all encoder hidden states dynamically at every step.

Instead of forcing a single context vector c , Attention computes a unique context vector c_i for each decoder step i as a weighted sum:

$$c_i = \sum_{j=1}^T \alpha_{ij} h_j$$

where α_{ij} represents the alignment weight between decoder state s_{i-1} and encoder state h_j .

3 Computing Attention Weights: Step-by-Step Math

Alignment scores are converted to a probability distribution over encoder states using softmax.

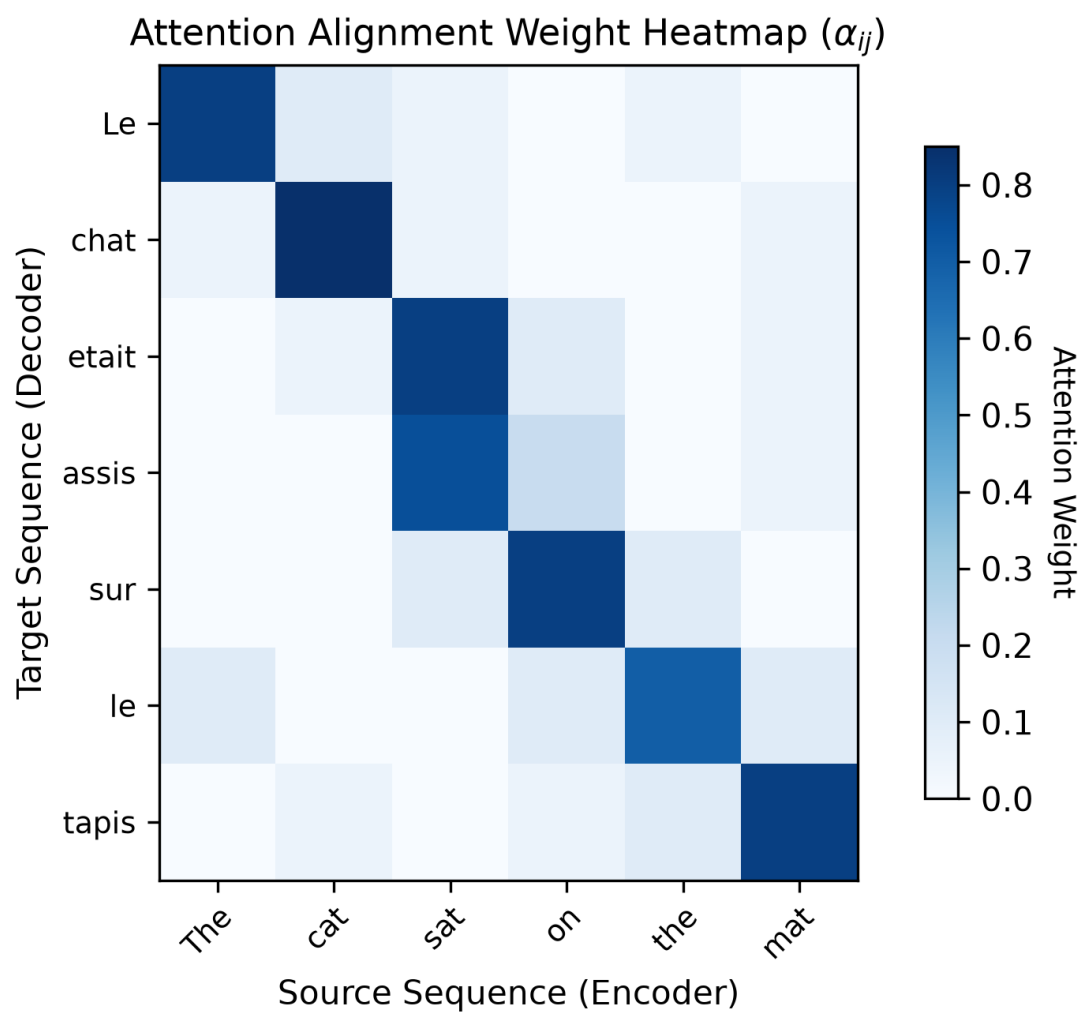


Figure 2: Attention alignment matrix α_{ij} showing soft word alignments between source and target sentences.

♣ Alignment Score Formulations

1. Bahdanau (Additive) Attention:

$$e_{ij} = v_a^T \tanh(W_a s_{i-1} + U_a h_j)$$

2. Luong (Multiplicative) Attention:

$$e_{ij} = s_{i-1}^T W_a h_j \quad \text{or simple dot product } e_{ij} = s_{i-1}^T h_j$$

📎 Full Worked Attention Numerical

Suppose an encoder has $T = 3$ hidden states:

$$h_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad h_2 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \quad h_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

The decoder state is $s_{i-1} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. We use Dot-Product Attention $e_{ij} = s_{i-1}^T h_j$.

Step 1. Compute raw alignment scores e_{ij} :

$$e_{i1} = \begin{bmatrix} 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 2.0, \quad e_{i2} = \begin{bmatrix} 2 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = 2.0$$

$$e_{i3} = \begin{bmatrix} 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 3.0$$

Step 2. Apply Softmax to compute attention weights α_{ij} :

$$\exp(e_{i1}) = 7.389, \quad \exp(e_{i2}) = 7.389, \quad \exp(e_{i3}) = 20.086$$

$$\sum_{j=1}^3 \exp(e_{ij}) = 7.389 + 7.389 + 20.086 = 34.864$$

$$\alpha_{i1} = 0.2119, \quad \alpha_{i2} = 0.2119, \quad \alpha_{i3} = 0.5761$$

Step 3. Calculate dynamic context vector c_i :

$$\begin{aligned} c_i &= 0.2119 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0.2119 \begin{bmatrix} 0 \\ 2 \end{bmatrix} + 0.5761 \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0.2119 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.4238 \end{bmatrix} + \begin{bmatrix} 0.5761 \\ 0.5761 \end{bmatrix} = \begin{bmatrix} 0.7880 \\ 0.9999 \end{bmatrix} \end{aligned}$$

✓ Additive vs Multiplicative Attention

Property	Bahdanau (Additive)	Luong (Multiplicative)
Score Formula	$v_a^T \tanh(W_a s + U_a h)$	$s^T W_a h$
Parameters	Requires W_a, U_a, v_a	Requires only W_a (or 0 for dot)
Complexity	Higher	Lower (optimized BLAS dot products)

4 Self-Test & Pocket Recap

Test your knowledge on attention mechanisms and context computation.

Self-Test Questions

1. **Question:** Why does Luong attention scale better computationally than Bahdanau attention?

Answer: Luong attention uses matrix dot products ($s^T W_a h$) which map directly to highly optimized GPU matrix multiplication routines, whereas Bahdanau attention requires non-linear activation functions (tanh).

2. **Question:** If all alignment scores e_{ij} are equal to 5.0 for sequence length $T = 4$, what are the attention weights?

Answer: Since all raw scores are equal, softmax produces a uniform distribution: $a_{ij} = \frac{1}{4} = 0.25$ for all $j = 1, 2, 3, 4$.